CPAD Lab 03 — Viva Defense Guide

A plain-English walkthrough of exactly what was built, why, and how to talk about it confidently in a viva — written for Chaitanya to read and understand his own submission.

How to use this document: read Part 1 once to get the big picture. Skim Part 2 so you know which file does what. Read Part 3 properly — it explains every technique line by line in plain words. Rehearse Part 4 out loud before the viva; those are the exact kinds of questions you'll be asked.

Part 1 — The Big Picture (30-second summary)

Your Lab 02 app was an Expense Tracker that only ever looked right on a phone. Lab 03's job was to make it also look right on a tablet or a wide browser window, and to react correctly when the phone is rotated sideways. You did this with four Flutter tools, and you did NOT rewrite the app — you wrapped the existing screen in logic that decides, at the moment it draws itself, which version of the layout to show.

The one-sentence answer if you're ever asked "what did you do in this lab": *"I made HomeScreen check how much width it has using LayoutBuilder, and if it's 600 dp or wider I show a two-pane tablet layout instead of the single-column phone layout, and inside that tablet layout I used OrientationBuilder so the grid re-flows from 2 to 3 columns when you rotate the device."*

The two core ideas you must never mix up

	Adaptive design	Responsive design
What it means	Show a DIFFERENT layout depending on device class	Let ONE layout fluidly stretch/shrink to fit any width
In your app	The LayoutBuilder switch between _mobileLayout() and _tabletLayout() — two genuinely different widget trees	Expanded and Wrap inside each of those layouts, so they don't overflow as the width changes a bit
Real-world analogy	A website that shows a hamburger menu on mobile and a full nav bar on desktop — structurally different	A photo that resizes to fit its frame — same picture, just scaled

Part 2 — What Changed, File by File

Six of the ten files in this project are 100% untouched from Lab 02 (main.dart, expense.dart, expense_provider.dart, storage_service.dart, add_expense_screen.dart, expense_card.dart). If asked about any of those, you can honestly say “that's the same as Lab 02, I didn't touch it for this experiment.” Only four files matter for Lab 03:

1. home_screen.dart — the main change

- Added a LayoutBuilder as the very first thing inside the Scaffold's body.
- It reads constraints.maxWidth (the width the parent gave this widget — not the phone's total screen width, an important distinction, see Part 4).
- If that width is ≥ 600, it calls a new method _tabletLayout(); otherwise it calls _mobileLayout().
- _mobileLayout() is exactly your old Lab 02 CustomScrollView — nothing inside it changed.
- _tabletLayout() is brand new: a Row with two children — a fixed-width SizedBox on the left holding the summary (total card, stats, chart) inside its own scroll view, and an Expanded on the right holding the “Recent Transactions” header plus the new expense grid.
- Inside _tabletLayout(), MediaQuery.of(context).orientation is read once to decide if the left pane should be 380 or 320 wide.

2. widgets/responsive_expense_grid.dart — a brand new file

- This is a small new widget: it takes a list of expenses and a crossAxisCount number, and lays them out in a GridView instead of a vertical list.
- It reuses your existing ExpenseCard widget for each cell — so a card looks the same whether it's in the mobile list or the tablet grid.
- The crossAxisCount (2 or 3) is decided by the caller (home_screen.dart), not by this file — this file just draws whatever count it's told.

3. widgets/stats_row.dart — small addition

- Lab 02 already had 4 stat cards (Today / This Week / This Month / Avg) in a Row of 4 Expanded widgets.
- Lab 03 wraps that Row in a LayoutBuilder. If the available width drops below 360 dp (a very narrow phone, or the tablet's narrow left pane in an extreme case), it switches to a Wrap instead — 2 cards per line — so the cards don't get squashed unreadably thin.

4. widgets/chart_section.dart — small addition

- Lab 02 already drew a pie chart at a fixed height of 200.
- Lab 03 wraps the chart in an OrientationBuilder so the height becomes 160 in landscape (freeing up vertical space) and stays 200 in portrait.

Part 3 — The Four Techniques, Explained Simply

MediaQuery — “what kind of device/screen am I on?”

MediaQuery.of(context) gives you information about the whole device: its total size, orientation, text scale factor, whether it's in dark mode, etc. In this app it is used exactly once — to read .orientation — to decide how wide the left pane of the tablet layout should be (380 in portrait vs 320 in landscape).

```
final isLandscape = MediaQuery.of(context).orientation == Orientation.landscape;
```

Plain English: “Ask the phone/tablet whether it's currently being held sideways.”

LayoutBuilder — “how much space did MY PARENT give me?”

This is subtly different from MediaQuery. LayoutBuilder does not look at the whole device screen — it looks at the constraints the immediate parent widget has handed down. In a simple full-screen scaffold body the two numbers are usually the same, but they are conceptually different, and this difference is a favourite viva trap.

```
LayoutBuilder(
  builder: (context, constraints) {
    final isTablet = constraints.maxWidth >= 600;
    ...
  },
)
```

Plain English: “Before I decide how to draw myself, let me check how many pixels of width I've actually been allocated by whatever contains me.” Used twice in this app: once in home_screen.dart for the mobile/tablet switch, once in stats_row.dart for the 360dp Wrap fallback.

OrientationBuilder — “rebuild me specifically when the phone rotates”

This is a shortcut for reacting to portrait ↔ landscape changes. Technically it's built out of the same idea as LayoutBuilder (it also looks at constraints and infers orientation from whether width or height is bigger), but it reads more clearly in code when all you care about is portrait-vs-landscape rather than an exact pixel number.

```
OrientationBuilder(
  builder: (context, orientation) {
    final crossAxisCount = orientation == Orientation.portrait ? 2 : 3;
    ...
  },
)
```

Used twice: around the expense grid (2 columns portrait → 3 columns landscape) and around the pie chart (200px tall portrait → 160px tall landscape, to save vertical space when the screen is short).

Expanded, Flexible, and Wrap — “don't let things overflow”

These solve the classic Flutter error every student hits: “A RenderFlex overflowed by 40 pixels.” Row and Column give their children exactly the size the child asks for, which breaks the moment content doesn't fit. These three widgets fix that:

- Expanded — “take up all the leftover space in this Row/Column, divided by flex.” Used for the tablet's right pane, and for each of the 4 stat cards.
- Flexible — the general-purpose parent class of Expanded. Expanded is just Flexible with fit locked to FlexFit.tight (must fill the space). Flexible on its own (FlexFit.loose) would let a child be smaller than its share if it doesn't need all of it. This app only needed the “always fill” behaviour, so Expanded was used everywhere — but because Expanded literally extends Flexible in Flutter's own source code, you ARE using Flexible's mechanism, just via its stricter subclass.
- Wrap — “if children don't fit on one line, flow the overflow onto a new line” — like text wrapping, but for widgets. Used for the category legend under the pie chart, and for the stats cards on very narrow screens.

Part 4 — Viva Rehearsal (read these out loud)

Category A — Conceptual questions

Q: What's the difference between adaptive and responsive design?

A: Responsive means one layout that fluidly stretches — same widget tree, different size. Adaptive means switching to a structurally different layout at a breakpoint. My app does both: LayoutBuilder switches between two different widget trees (adaptive) at 600dp, and inside each of those, Expanded and Wrap let things resize fluidly without breaking (responsive).

Q: Why 600dp specifically?

A: 600dp is Google's own Material Design guideline for the phone/tablet boundary — it's the same number used by Android's own resource qualifiers (sw600dp). I used it as a constant, tabletBreakpoint = 600.0, so it's easy to find and change in one place.

Q: What's the exact difference between MediaQuery and LayoutBuilder?

A: MediaQuery reads properties of the whole device/screen — total size, orientation, brightness, text scale. LayoutBuilder reads the constraints passed down by this widget's specific parent, which can be smaller than the full screen if this widget is nested inside something else (a dialog, a side panel, a column that only gave it half the width). In my app's case they'd usually agree because HomeScreen's body fills the whole scaffold, but if I nested my tablet layout inside, say, a master-detail panel that only gave it 400dp of the 1000dp screen, LayoutBuilder would correctly say 400 while MediaQuery would still say 1000.

Q: Why did you use OrientationBuilder instead of just using LayoutBuilder everywhere?

A: I could have used LayoutBuilder and compared width vs height myself, but OrientationBuilder is a purpose-built shortcut for exactly that comparison, and it makes the code more readable when the only thing I care about is portrait vs landscape, not an exact pixel value.

Category B — Code-specific questions

Q: Walk me through what happens when I resize the browser window past 600px.

A: The Scaffold body is wrapped in a LayoutBuilder. Every time the available width changes, Flutter re-runs that builder function. It checks constraints.maxWidth >= 600. The moment it crosses 600, the builder returns _tabletLayout(...) instead of _mobileLayout(...), and Flutter rebuilds that whole subtree — you'd see the single column snap into the two-pane view.

Q: What's inside _tabletLayout()?

A: A Row with two children. The left child is a SizedBox with a fixed width (380 portrait / 320 landscape) containing a CustomScrollView with the total card, stats row, and chart. The right child is an Expanded containing a header and an OrientationBuilder that renders the expense grid with either 2 or 3 columns.

Q: Why is the grid its own separate widget file instead of inline in home_screen.dart?

A: Separation of concerns — ResponsiveExpenseGrid only needs to know 'here's a list and a column count, draw a grid.' It doesn't need to know anything about LayoutBuilder or OrientationBuilder; the caller decides the column count and just passes in a plain integer. That makes the grid widget reusable and easy to test on its own.

Q: What does ExpenseCard have to do with the grid?

A: ResponsiveExpenseGrid doesn't draw its own cards from scratch — it reuses the exact same ExpenseCard widget that the mobile list already used in Lab 02. That's why a card looks identical whether you're looking at the phone list or the tablet grid; only the container around it (ListView vs GridView) is different.

Q: What happens if the expense list is empty in the tablet layout?

A: There's an explicit if (expenses.isEmpty) check inside the OrientationBuilder that returns a centred icon + 'No expenses yet. Tap + to add one.' message instead of an empty GridView — the same empty-state message used on mobile.

Category C — “What if” / trick questions

Q: What if I asked you to change the breakpoint to 500dp right now, live?

A: I'd change one line: static const double tabletBreakpoint = 600.0; to 500.0, inside HomeScreen. That's the only place the number is used for the layout switch.

Q: What if I asked you to add a 4th column on very wide desktop screens (say >1200dp)?

A: Inside the OrientationBuilder in home_screen.dart, instead of a plain orientation ? 2 : 3 ternary, I'd check constraints.maxWidth as well — e.g. if width > 1200 use 4, else use the existing portrait/landscape logic. I'd need to pass constraints down into that builder, which it already has access to since it's inside _tabletLayout(context: ..., constraints: ...).

Q: Is Expanded the same as Flexible in your code?

A: Not exactly the same widget, but Expanded IS a Flexible under the hood — in Flutter's own source, class Expanded extends Flexible and just hard-codes fit: FlexFit.tight. I used Expanded everywhere because I always wanted children to fill their full share; I never needed Flexible's looser 'shrink if you don't need the space' behaviour in this particular layout.

Q: Does your app work correctly with zero internet / zero backend?

A: Yes — storage_service.dart persists expenses locally (this was already true from Lab 02 and wasn't touched in Lab 03). Nothing about the responsive layout logic depends on network access; it's purely a function of the widget's constraints and orientation.

Part 5 — One Honest Thing You Should Know

During verification, one real bug was found in responsive_expense_grid.dart — it used the ExpenseCard widget without importing the file that defines it, which would have stopped the project from compiling. This has already been fixed (one import line added). If you're asked about it: this is a completely normal, common Dart mistake (forgetting an import for a class in a different file, even in the same folder) and is not a sign of a design flaw — it's a one-line fix. Knowing about it and being able to explain it calmly is actually a good sign in a viva; it shows you understand how Dart's import system works.

```
// Before (broken — ExpenseCard used but never imported):
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import '../models/expense.dart';

// After (fixed):
import 'package:flutter/material.dart';
import '../models/expense.dart';
import 'expense_card.dart';   // <-- the missing line
```

If asked “why was intl removed too?” — it was an unused import (NumberFormat/DateFormat from that package was never actually called in this file), so it was just dead weight; removing it doesn't change behaviour, it just keeps the file clean.

Part 6 — Quick Cheat Sheet

Tool	Answers the question…	Used in this app for…
MediaQuery	What device/screen am I on overall?	Reading orientation to size the tablet left pane (380/320)
LayoutBuilder	How much space did my parent give ME?	600dp mobile/tablet switch; 360dp StatsRow Wrap fallback
OrientationBuilder	Am I portrait or landscape right now?	Grid columns 2 ↔ 3; chart height 200 ↔ 160
Expanded	How do I fill leftover Row/Column space?	Tablet right pane; each of the 4 stat cards
Wrap	What if my children don't all fit on one line?	Category legend chips; narrow-screen stat cards

Good luck — you built this, and now you know exactly how it works.